

W 4

HTML 網頁結構解析與資料擷取

DOM 結構與元素定位 · BeautifulSoup · 多筆擷取與檔案下載

吳彥霖 博士

Python 爬蟲實務應用 · 第 4 週 · 10/06

三節課，三件事

節 1

HTML 結構與元素定位

- DOM 樹：tag / class / id
- DevTools 元素面板定位
- CSS selector

節 2

BeautifulSoup 擷取資料

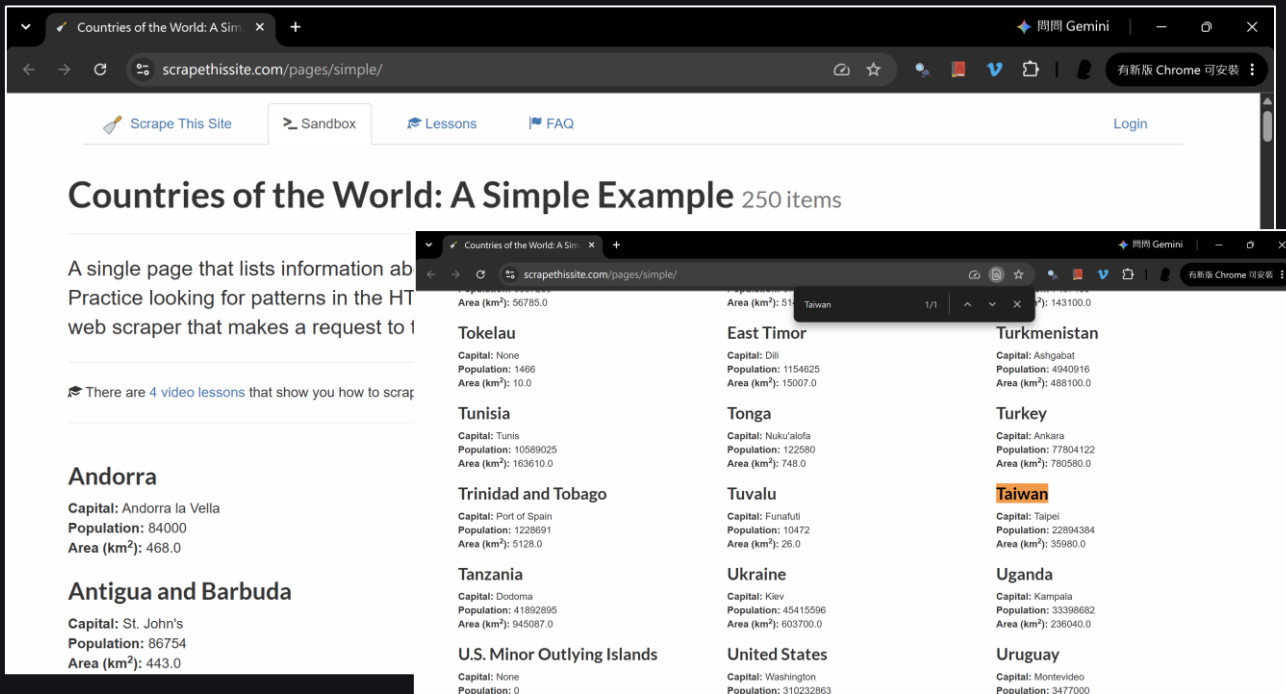
- find / find_all / select
- get_text / 取屬性

節 3

多筆擷取、存檔與下載

- list[dict] → CSV
- read_html vs BeautifulSoup

課堂練習 - 1



<https://www.scrapethissite.com/pages/simple/>

- 爬取 ScrapeThisSite 練習網站的 Taiwan 相關資訊

Step 1 : 解析伺服器回應的 Response

```
import requests

URL = "....."
HEADERS = {"User-Agent": "....."}

resp = requests.get(URL, headers=HEADERS, timeout=10)

# ----- Response ( 伺服器回應內容 ) -----
print(f"[Response] 狀態碼={resp.status_code} {resp.reason}")
print(f"[Response] Content-Type={resp.headers.get('Content-Type')}")
print(f"[Response] 編碼={resp.encoding}")
print(f"[Response] 花費秒數={resp.elapsed.total_seconds():.2f}")
print(f"[Response] User-Agent={resp.request.headers['User-Agent']}")
```

Step 2 : 取得且解析網頁原始碼

Step 3 : 轉換網頁表格成 DataFrame 清單

Step 4 : DataFrame 寫入 Excel

為什麼爬取不到資料？

網頁	資料長什麼樣子	pd.read_html	該用什麼
Hockey Teams	<table> 表格	可以	read_html 一行搞定
Countries of the World	<div> 卡片，一國一張	No tables found	BeautifulSoup：自己定位每個欄位

問題 真實網頁的資料多半「長得像表格，卻不是 <table>」→ 要先看懂 HTML 結構，才能把資料拿出來

參考文件：

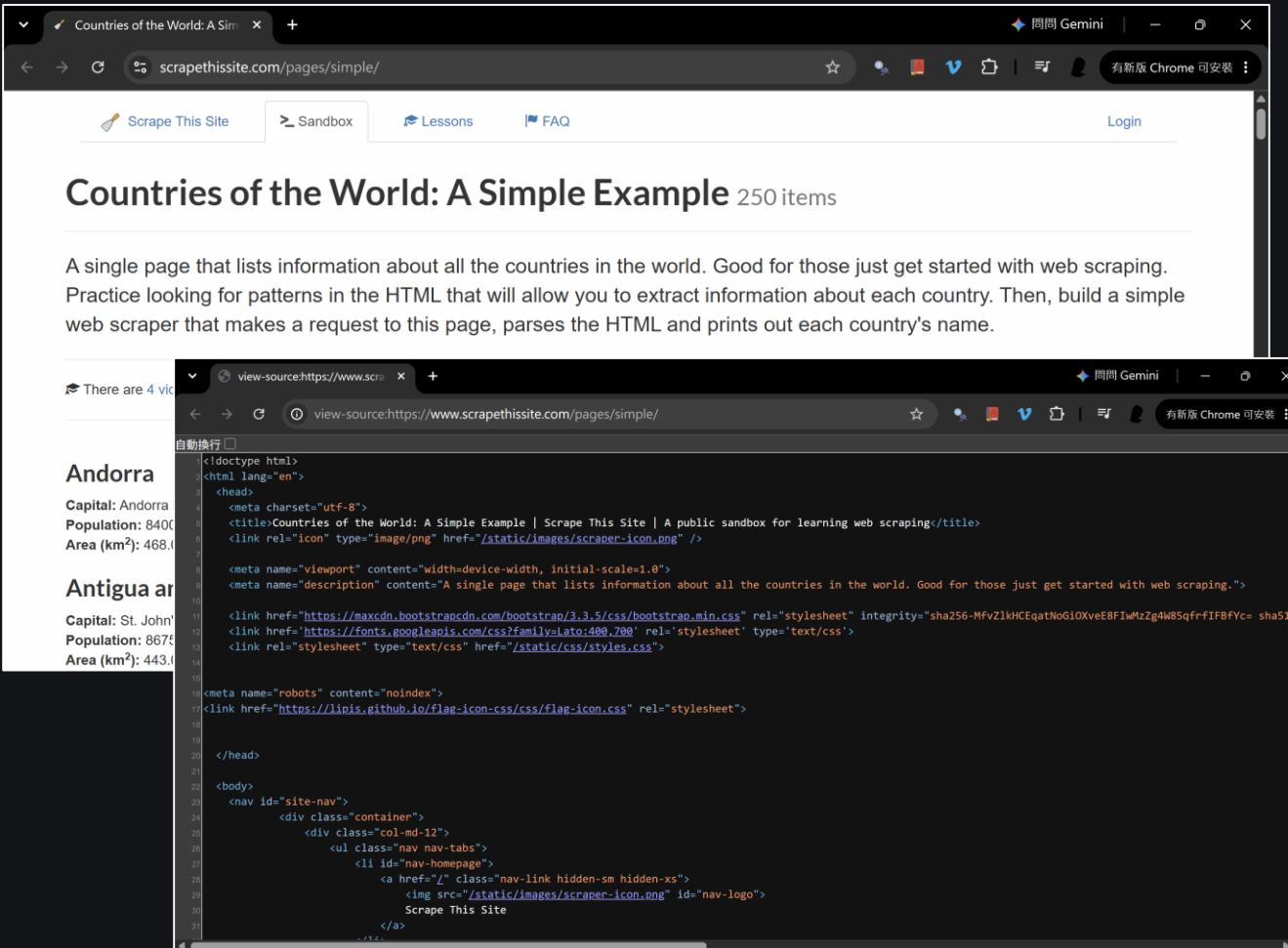
■ [pandas.read_html](#)

節 1

HTML 結構與元素定位

Document Object Model

- 當瀏覽器載入一個 HTML 網頁時，會將 HTML 程式碼解析，且在記憶體中建立一個樹狀結構，這個結構稱為 DOM 樹 (DOM Tree)。
- DOM 樹為網頁物件化地圖，將網頁上的每一個元素、文字、甚至屬性，皆變成一個個可被 JavaScript 操控的物件(Node)。



Document Object Model

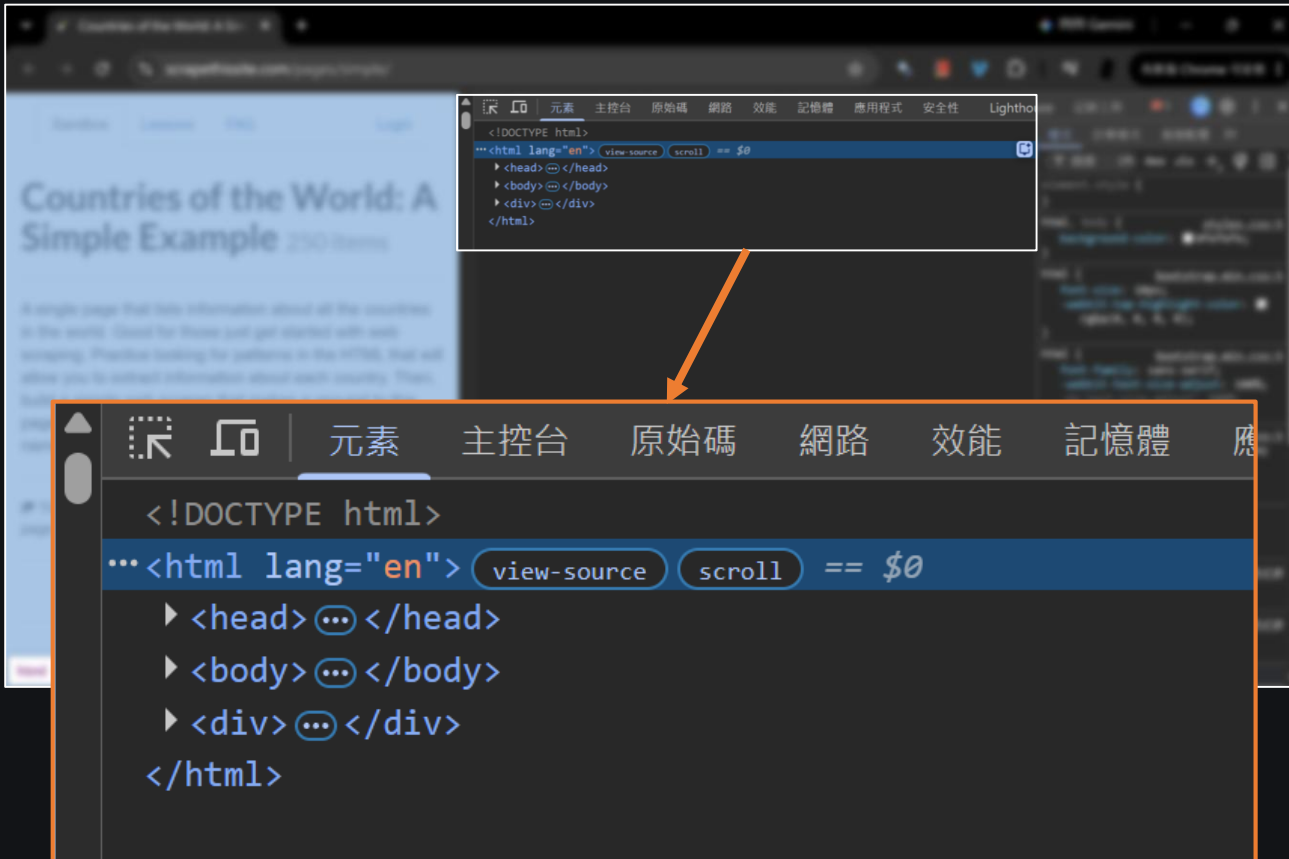
● DOM 結構

document

```
├── <!DOCTYPE html>  
└── html
```

← 根節點 (soup)
← 宣告 HTML 文件
← 根目錄

```
├── head  
│   ├── meta charset="utf-8"  
│   └── title  
│       └── "Countries of the World..."  
└── body  
    ├── section id="countries"  
    │   ├── div class="col-md-4 country"  
    │   │   ├── h3 class="country-name"  
    │   │   │   └── "Andorra"  
    │   │   └── div class="country-info"  
    │   │       ├── span class="country-capital"  
    │   │       │   └── "Andorra la Vella"
```



Document Object Model

● DOM 結構

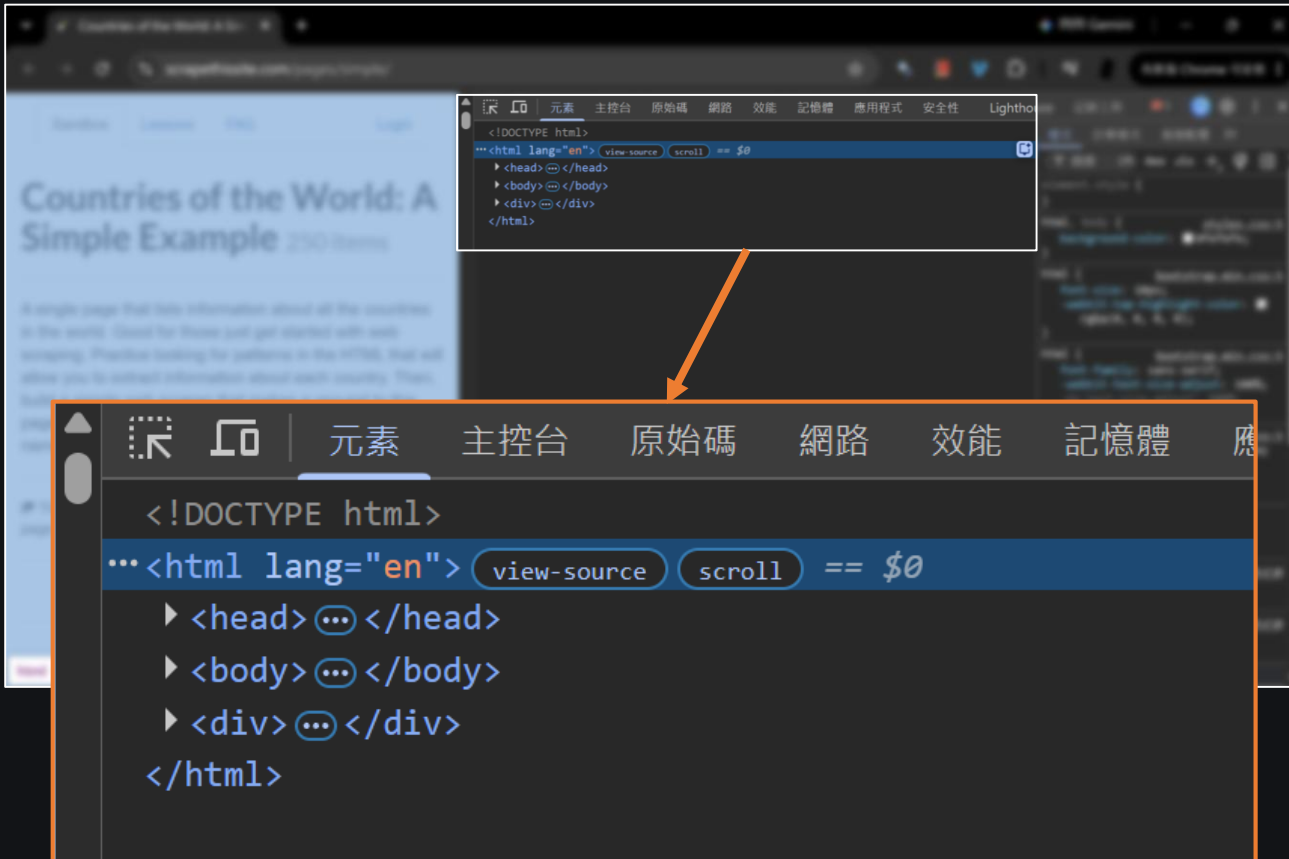
document

├─ <!DOCTYPE html>
└─ html

← 根節點 (soup)
← 宣告 HTML 文件
← 根目錄

├─ head
│ ├── meta charset="utf-8"
│ └── title
│ └── "Countries of the World..."
└─ body

網頁的大腦，放置給瀏覽器與搜尋引擎檢視的設定，
例如：網頁標題 <title>
編碼 <meta charset="utf-8">

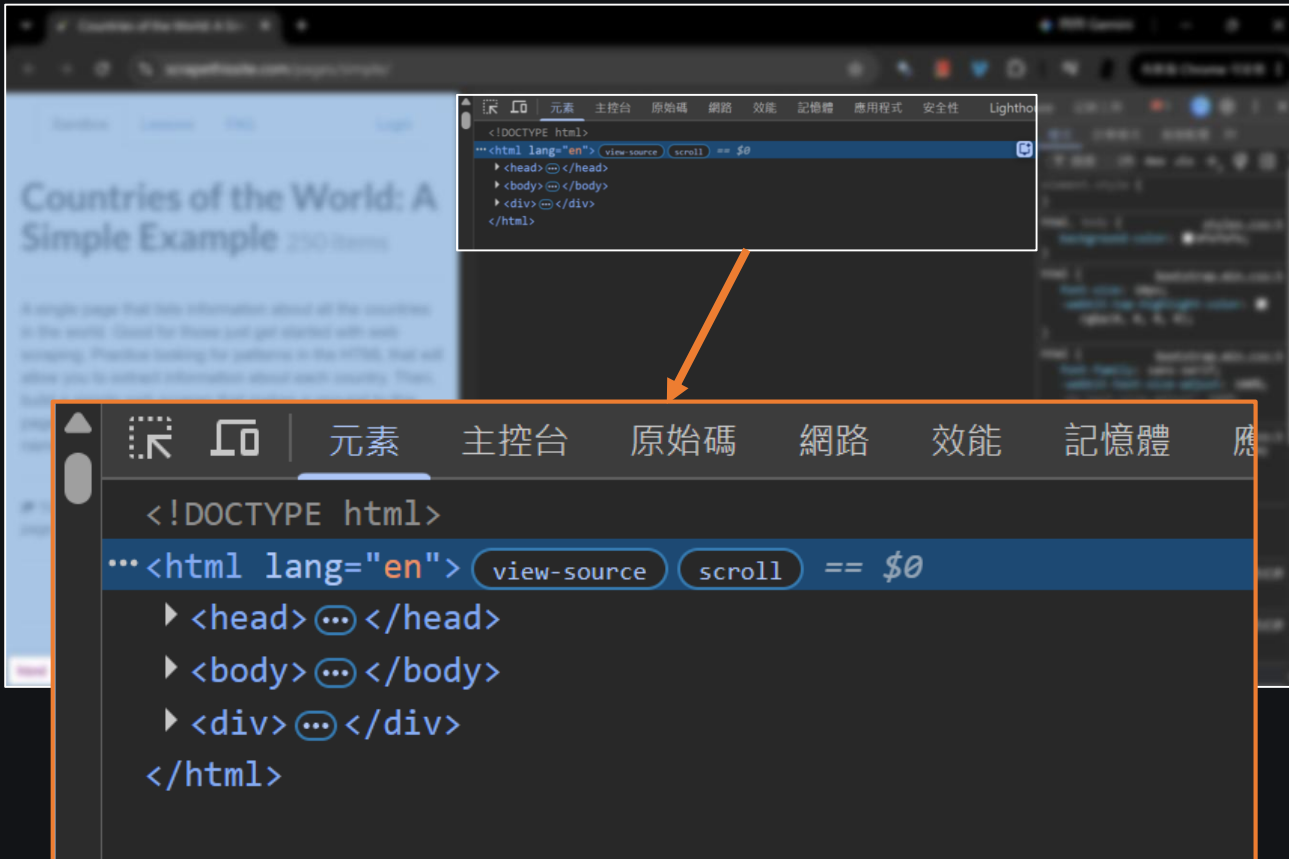


Document Object Model

● DOM 結構



網頁的身體，用來放置使用者實際在頁面上能看見的文字、圖片或互動內容。



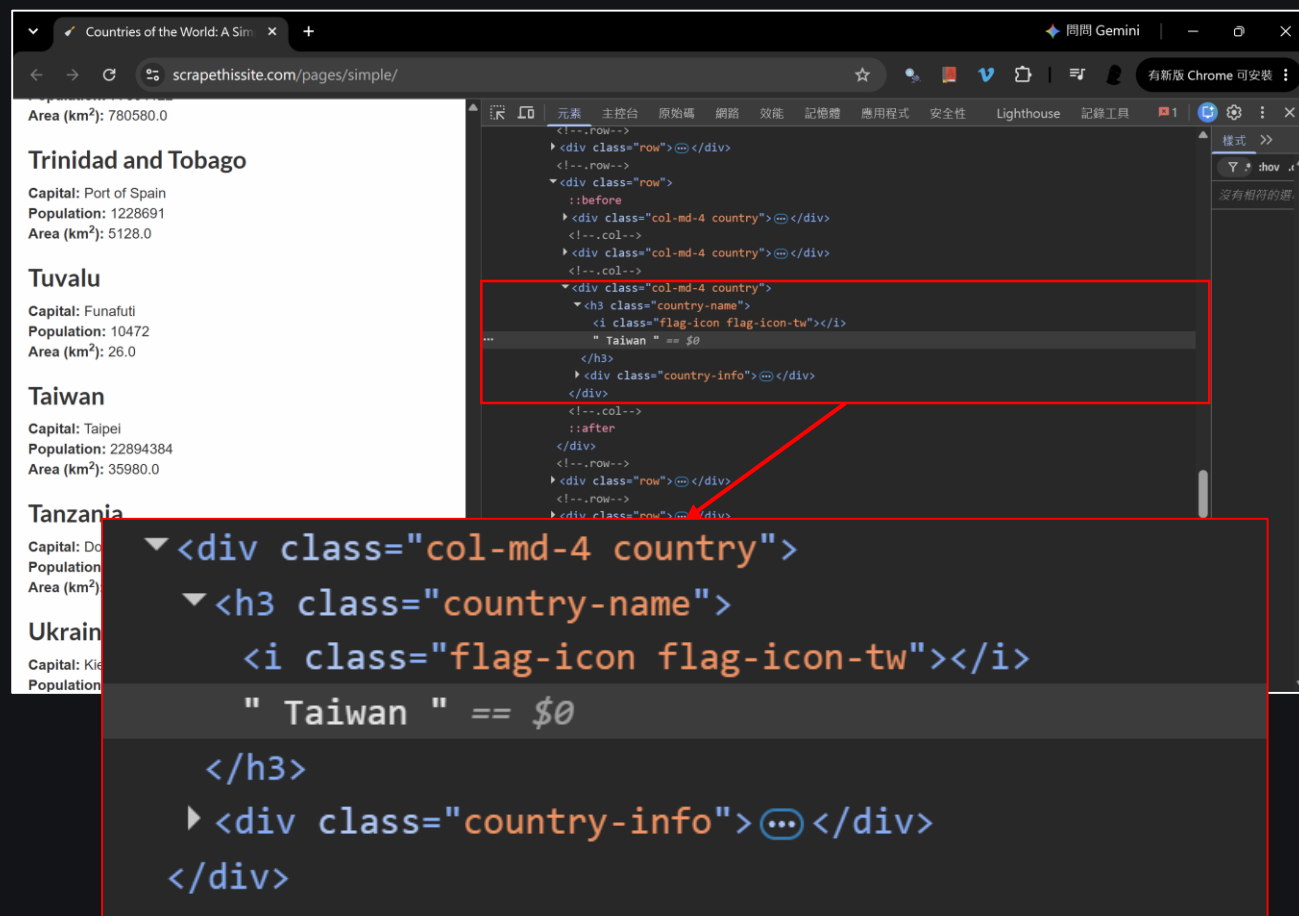
一個元素的組成

- 元素 = 起始標籤 + 屬性 + 內容 + 結束標籤

h3 **class** Taiwan **/h3**

定位線索

- div 區塊、分隔容器
- h3 第三級標題標籤
- i 文字字體



一個國家 = 一張卡片

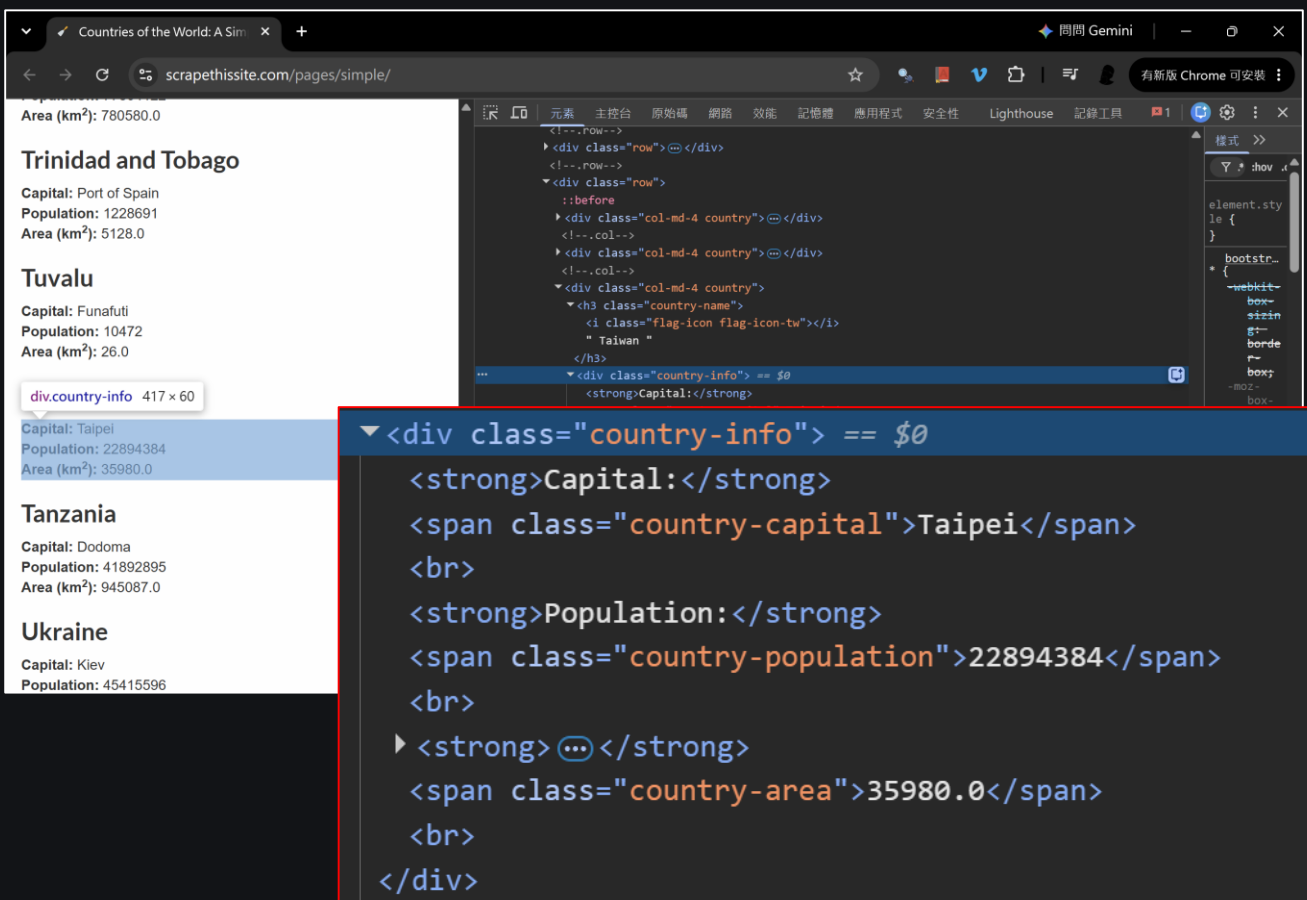
一個元素的組成

- 元素 = 起始標籤 + 屬性 + 內容 + 結束標籤

h3 **class** Taiwan **/h3**

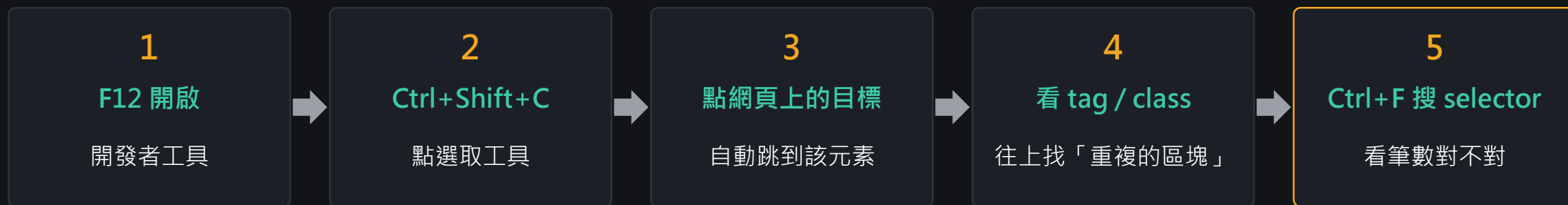
定位線索

- div 區塊、分隔容器
- h3 第三級標題標籤
- i 文字字體



首都、人口、面積資訊

用開發者工具定位元素



結論 先在瀏覽器驗證 selector，再撰寫 Python 程式，可省下大量「為什麼抓不到」的除錯時間。

課堂練習 - 1

在 [Countries of the World](#) 中尋找出 5 個欄位的 CSS selector，填進 week_04_exercise_1.py 執行驗證。

week_04_exercise_1.py

```
SELECTORS = {  
    "國家卡片": "?",  
    "國名": "?",  
    "首都": "?",  
    "人口": "?",  
    "面積": "?",  
}  
# 程式會自動檢查：每個欄位都要抓到 250 筆
```

預期輸出

```
[OK] 國家卡片 ? 抓到 250 筆  
[OK] 國名      ? 抓到 250 筆  
[OK] 首都      ? 抓到 250 筆  
[OK] 人口      ? 抓到 250 筆  
[OK] 面積      ? 抓到 250 筆
```

今天，我們學到了什麼？

下週見 · Python 爬蟲實務應用